

Alpha Inmobiliaria — Contexto Completo del Proyecto

Documento de contexto para ser usado por modelos de IA. Contiene todo lo necesario para entender el proyecto: objetivos, equipo, stack técnico, arquitectura, modelo de datos, historias de usuario, tareas técnicas y estado actual.

1. Resumen del Proyecto

- Nombre:** Alpha Inmobiliaria
- Tipo:** Plataforma web de bienes raíces
- Propósito:** Permitir a asesores inmobiliarios publicar, gestionar y promocionar propiedades en venta/alquiler, y a clientes buscar, filtrar, ver mapas interactivos y contactar asesores mediante un chatbot con IA.
- Metodología:** SCRUM con Kanban, MoSCoW para priorización, Fibonacci para estimación.
- Sprints:** 3 Sprints de 2 semanas cada uno.
- Historias de Usuario:** 32 historias, 162 puntos total.
- Monorepo:** `apps/backend` (Express + TypeScript), `apps/frontend-public` (pendiente), `apps/frontend-admin` (pendiente).

2. Stack Tecnológico

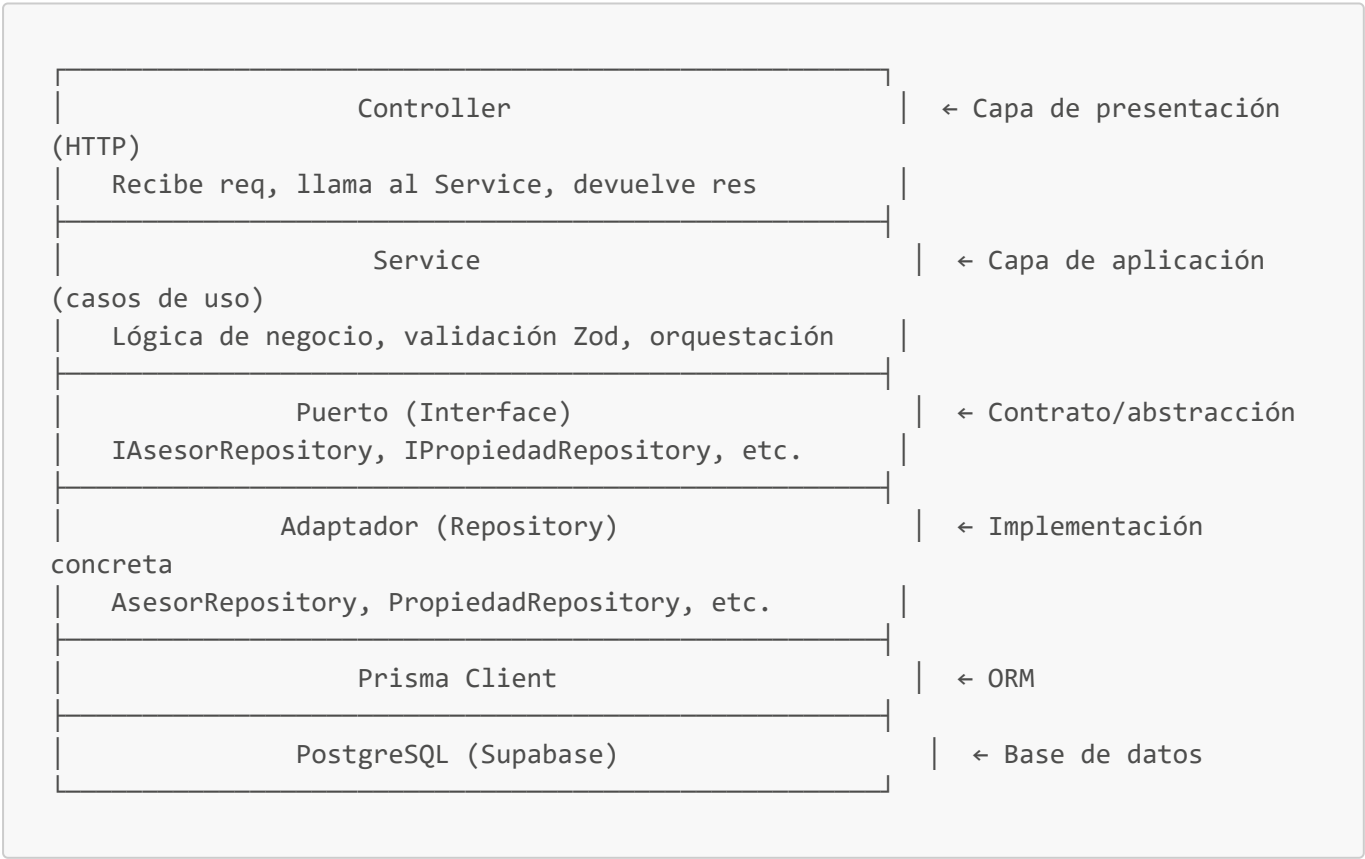
Capa	Tecnología	Versión
Runtime	Node.js	22+
Lenguaje	TypeScript	6.x
Framework Backend	Express	5.x
ORM	Prisma	7.x (driver adapter <code>@prisma/adapter-pg</code>)
Base de Datos	PostgreSQL (Supabase)	15+
Validación	Zod	4.x
Autenticación	JWT (pendiente)	—
Chatbot	GROQ API (gratuito)	—
Mapas	Leaflet / Mapbox (pendiente)	—
Almacenamiento Imágenes	Cloudinary (gratuito)	—
Frontend Público	React + Vite (pendiente)	—
Frontend Admin	React + Vite (pendiente)	—

Capa	Tecnología	Versión
Despliegue	Render (Backend), Vercel (Frontend)	—

3. Equipo y Roles

Miembro	Rol Principal	Responsabilidades
Yandri	Backend Lead / DevOps	Configuración del monorepo, Prisma schema, conexión DB, CRUD Propiedades, CRUD Asesores, API Mensajes, autenticación JWT, despliegue. Ejecuta todas las TTs técnicas del backend.
Jhonny	Frontend Público	Catálogo de propiedades, búsqueda y filtros, detalle de propiedad, formulario de contacto, favoritos, compartir, programar citas, notificaciones, SEO.
Kelvin	Admin Panel	CRUD propiedades (panel), galería de imágenes, panel de administración, imágenes desde dispositivo, reportes, tours virtuales, exportar datos.
Allan	Frontend / Mapas	Mapa interactivo, perfil de asesor, testimonios, responsive design, reset de contraseña, valoraciones, términos, Google Maps, modo oscuro.
Ricardo	Chatbot IA	Chatbot con GROQ API, chat en vivo, sugerencias IA avanzadas.

4. Arquitectura — Hexagonal (Puertos y Adaptadores)



Flujo de una petición típica (con Inversión de Dependencias)

```

POST /api/asesores { nombre, email, password, telefono }
  → routes/asesor.routes.ts           ← Wiring: Repo → Service →
Controller
  → AsesorController.crear(req.body)   ← Solo conoce AsesorService
(abstracto)
  → AsesorService.crear(data)         ← Solo conoce IAsesorRepository
(puerto)
  → createAsesorSchema.parse(data)    ← Zod validation
  → this.repository.findByEmail(email) ← Verifica duplicado
  → bcrypt.hash(password, 10)         ← Hashea contraseña
  → this.repository.create({...data})  ← Transacción Prisma
    → User.create() → Asesor.create() ← DB
  ← res.status(201).json(asesor)

```

Inyección de Dependencias: Las dependencias se inyectan por constructor en el punto de wiring (routes), nunca se importan directamente desde capas superiores. Esto permite testear con mocks y cambiar implementaciones sin modificar el código de negocio.

5. Modelo de Datos (7 Entidades, 5 Enums)

Enums

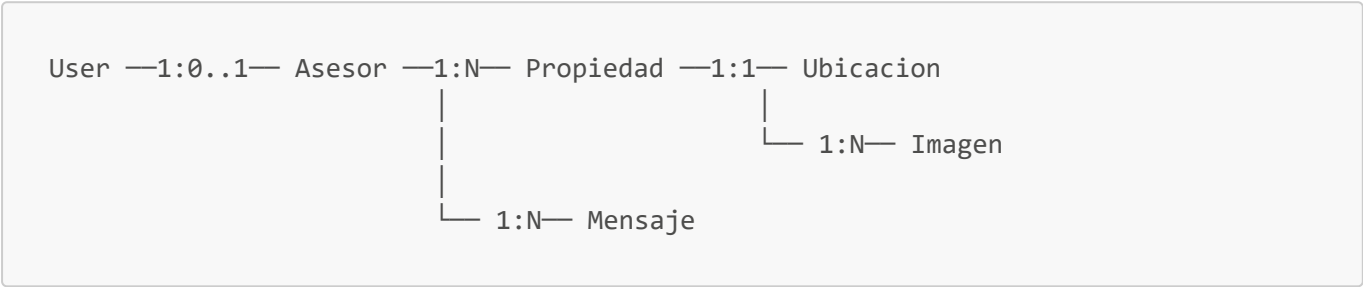
Enum	Valores
Rol	asesor, admin
TipoPropiedad	casa, departamento, terreno, local, oficina
TipoTransaccion	venta, alquiler
EstadoPropiedad	activa, pausada, vendida, alquilada
TipoMensaje	contacto, venta

Entidades y Relaciones

Entidad	Campos Clave	FK
User	id, email (único), password (hash), rol, activo	—
Asesor	id, nombre, telefono, foto?, especialidad?, descripcion?, añosExperiencia?	userId → User
Propiedad	id, titulo, descripcion, precio, tipoPropiedad, tipoTransaccion, habitaciones?, baños?, parqueos?, metrajeTotal?, metrajeConstruido?, estado, destacada	asesorId → Asesor, ubicacionId → Ubicacion
Ubicacion	id, direccion, sector?, ciudad (default: "Guayaquil"), provincia (default: "Guayas"), latitud, longitud	← Propiedad (1:1)

Entidad	Campos Clave	FK
Imagen	id, url, publicId, orden	propiedadId → Propiedad
Mensaje	id, nombre, email, telefono?, mensaje, tipo, leído, archivado	propiedadId? → Propiedad, asesorId → Asesor

Diagrama de Relaciones



Nota SOLID: User y Asesor están separados — User maneja solo autenticación (email, password, rol), Asesor maneja datos profesionales (nombre, teléfono, especialidad). Esto sigue el Principio de Responsabilidad Única.

6. Historias de Usuario (32) — Asignadas por Miembro

Estimadas con Fibonacci (1, 2, 3, 5, 8). Fuente: [Historias_de_Usuario_26-6-26.pdf](#) (32 historias consolidadas de 41 originales).

Sprint 1 — 18 historias, 82 pts

ID	Historia	Pts	Miembro
HU-001	Registro de usuario	3	Allan
HU-002	Inicio de sesión	3	Yandri
HU-003	Catálogo de propiedades	5	Jhonny
HU-004	Búsqueda y filtros	8	Jhonny
HU-005	Detalle de propiedad	5	Jhonny
HU-006	Gestión de propiedades (CRUD)	8	Kelvin
HU-007	Galería de imágenes	5	Kelvin
HU-008	Mapa interactivo	8	Allan
HU-009	Chatbot IA (GROQ)	8	Ricardo
HU-010	Formulario de contacto	3	Jhonny
HU-011	Perfil de asesor	5	Allan
HU-012	Panel de administración	8	Kelvin

ID	Historia	Pts	Miembro
HU-013	Notificaciones de mensajes	5	Jhonny
HU-014	Reset de contraseña	3	Allan
HU-015	Propiedades destacadas	3	Jhonny
HU-016	Testimonios	3	Allan
HU-017	Imágenes desde dispositivo	5	Kelvin
HU-018	Responsive design	5	Allan

Sprint 2 — 8 historias, 44 pts

ID	Historia	Pts	Miembro
HU-019	Autenticación JWT	8	Yandri
HU-020	Favoritos del usuario	5	Jhonny
HU-021	Compartir propiedad	3	Jhonny
HU-022	Reportes de propiedades	5	Kelvin
HU-023	Programar citas	8	Jhonny
HU-024	Valoraciones de asesores	3	Allan
HU-025	Términos y condiciones	3	Allan
HU-026	Configuración de perfil (asesor)	5	Yandri
HU-027	Chat en vivo	8	Ricardo

Sprint 3 — 6 historias, 36 pts

ID	Historia	Pts	Miembro
HU-028	Notificaciones push	5	Jhonny
HU-029	Exportar datos	3	Kelvin
HU-030	Integración Google Maps	5	Allan
HU-031	Tours virtuales	8	Kelvin
HU-032	Sugerencias IA (chatbot avanzado)	8	Ricardo
HU-033	Despliegue a producción	5	Yandri
HU-034	SEO	3	Jhonny
HU-035	Modo oscuro	3	Allan

Resumen por Miembro

Miembro	Sprint 1	Sprint 2	Sprint 3	Total Pts	# Historias
Yandri	3	13	5	21	3
Jhonny	26	16	8	50	9
Kelvin	26	5	11	42	7
Allan	24	6	8	38	7
Ricardo	8	8	8	24	3

7. Tareas Técnicas (TTs) — Backend

TTs ejecutadas por Yandri. Cada TT crea su documentación en *docs/TT-XXX-nombre.md*. Flujo Git: *feature/yandri-tt-00X-nombre* → PR → merge a *main*.

TT	Nombre	Estado	Archivos Clave
TT-001	Configuración del monorepo	✓	<i>package.json</i> , <i>apps/backend/tsconfig.json</i> , <i>apps/backend/src/server.ts</i>
TT-002	Esquema Prisma y modelo de datos	✓	<i>prisma/schema.prisma</i> , <i>prisma/migrations/</i>
TT-003	Conexión PostgreSQL (Supabase)	✓	<i>src/infrastructure/database/prisma.ts</i> , <i>.env</i>
TT-004	CRUD de Propiedades	✓	11 archivos (controller, service, repository, routes, validations, DTOs, ports)
TT-005	CRUD de Asesores	✓	9 archivos (controller, service, repository, routes, validation, DTOs, ports)
TT-005.5	Refactor de Arquitectura Hexagonal	✓	10 archivos modificados, 2 creados (entities)
TT-006	API de Mensajes + Cloudinary	🕒	Pendiente (MensajeController, MensajeService, Cloudinary)
TT-007	Autenticación JWT	📅	Pendiente (login, register, middleware, tokens)
TT-008	Búsqueda y filtros avanzados	📅	Pendiente (text search, price range, tipoPropiedad, tipoTransaccion)
TT-009	Integración de mapas	📅	Pendiente (Leaflet/Mapbox, geocoding)
TT-010	Chatbot con GROQ API	📅	Pendiente (chat endpoint, historial)

TT	Nombre	Estado	Archivos Clave
TT-011	Panel de administración		Pendiente (dashboard, stats)
TT-012	Frontend público		Pendiente (React + Vite, componentes)
TT-013	Despliegue y CI/CD		Pendiente (Render, Vercel, GitHub Actions)

Detalle de TTs Completadas

TT-001: Configuración del Monorepo

- Estructura `apps/backend`, `apps/frontend-public`, `apps/frontend-admin`
- TypeScript + tsx + Express 5
- Endpoint `/api/health`
- `.gitignore`, scripts `npm run dev`
- Git init + push a GitHub (`chaoslov/proyecto_bienesraices1`)

TT-002: Esquema Prisma

- `prisma-client` generator (Prisma 7)
- `postgresql` datasource
- 7 modelos: User, Asesor, Propiedad, Ubicacion, Imagen, Mensaje
- 5 enums: Rol, TipoPropiedad, TipoTransaccion, EstadoPropiedad, TipoMensaje
- `prisma validate` + `generate` exitoso

TT-003: Conexión PostgreSQL (Supabase)

- `@prisma/adapters-pg` + `pg` driver adapter
- Transaction Pooler + IPv4 add-on
- Parámetros de conexión pasados por separado (no URL string, evita parsing issues con `%40`)
- `import 'dotenv/config'` en `prisma.ts` para carga de variables de entorno
- Health check: `GET /api/health` → `{ "status": "ok", "db": "conectado" }`

TT-004: CRUD Propiedades

- 11 archivos siguiendo hexagonal architecture
- Zod validation (`createPropiedadSchema`, `updatePropiedadSchema`)
- 7 endpoints REST:
 - `GET /api/propiedades` — listar con filtros, paginación (`page`, `limit`, `total`)
 - `GET /api/propiedades/destacadas` — propiedades destacadas
 - `GET /api/propiedades/:id` — detalle con ubicación e imágenes
 - `POST /api/propiedades` — crear con ubicación anidada (transacción)
 - `PUT /api/propiedades/:id` — actualizar
 - `PATCH /api/propiedades/:id/estado` — cambiar estado
 - `DELETE /api/propiedades/:id` — eliminar

- DTOs: CreatePropiedadRequest, UpdatePropiedadRequest, FiltrosPropiedadRequest, PropiedadResponse
- Middleware `validateId.ts` con helper `getId(req)` para manejar `req.params.id`

TT-005: CRUD Asesores

- 9 archivos siguiendo hexagonal architecture
- Transacción Prisma: `User.create()` + `Asesor.create()` en `$transaction`
- Bcrypt para hash de contraseñas (10 salt rounds)
- 5 endpoints REST:
 - `GET /api/asesores` — listar (incluye `_count` de propiedades y mensajes)
 - `GET /api/asesores/:id` — detalle por ID
 - `POST /api/asesores` — crear (email único, password min 8 chars)
 - `PUT /api/asesores/:id` — actualizar perfil
 - `DELETE /api/asesores/:id` — elimina en cascada: mensajes → propiedades → asesor → user
- DTOs: CreateAsesorRequest, UpdateAsesorRequest, AsesorResponse

8. Decisiones Técnicas Clave

Decisión	Opción Elegida	Alternativa Descartada	Motivo
Base de Datos	Supabase	Neon	500 MB gratis, unificado Auth + Storage
ORM Driver	<code>@prisma/adapters-pg</code>	URL directa	Prisma 7 requiere driver adapter
Conexión	Pooler + IPv4 add-on	Directa	Pooler maneja múltiples conexiones mejor
User/Asesor	Separados (SOLID)	Unificados	User solo auth, Asesor datos profesionales
Chatbot	GROQ API (gratis)	OpenAI (pago)	Sin servidor, free tier generoso
tipoTransaccion	<code>venta</code> / <code>alquiler</code>	<code>comprar</code> / <code>alquilar</code>	Modelo RE/MAX, UI traduce "Comprar" → <code>venta</code>
Mensaje.tipo	<code>contacto</code> / <code>venta</code>	Solo contacto	Maneja consultas y solicitudes de venta
Hosting	Render + Vercel	Heroku (pago)	Free tier suficiente para el proyecto

9. Git Workflow

- **Repositorio:** `https://github.com/chaoslov/proyecto_bienesraices1.git`
- **Rama principal:** `main`
- **Ramas de features:** `feature/<nombre>-tt-00X-<descripcion>`
 - Ejemplo: `feature/yandri-tt-005-crud-asesores`

- **Flujo:** feature branch → Pull Request → code review → merge a **main**
 - **Commits:** Mensajes descriptivos en español
-

10. Estado Actual del Proyecto

Completado (Sprint 1, TT-001 a TT-005.5)

-  Monorepo configurado
-  Prisma schema con 7 entidades
-  Conexión PostgreSQL (Supabase)
-  CRUD Propiedades (7 endpoints)
-  CRUD Asesores (5 endpoints)
-  Refactor de Arquitectura Hexagonal — DI real, entidades de dominio, **implements**, eliminado acoplamiento a Prisma en puertos
-  Documentación de cada TT en **docs/**
-  Diagrama de clases completo en **docs/DIAGRAMA-CLASE-ENTIDADES.md**

En Progreso

-  TT-006: API de Mensajes + Cloudinary (siguiente tarea)

Pendiente (Sprint 1-3)

-  TT-006 a TT-013
-  Historias de usuario de frontend (Jhonny, Kelvin, Allan, Ricardo)

Prototipo en Producción

- URL: **<https://bienesyraicesgroupalpha.netlify.app/>** (prototipo visual, no conectado al backend)
-

11. Archivos Existentes

Raíz del proyecto

```
package.json      ← workspaces: ["apps/*"]
.gitignore
```

Backend (**apps/backend/**)

```
src/
├── server.ts      ← Express app, health check, router mount
├── infrastructure/
│   ├── database/prisma.ts ← Prisma client con adapter-pg
│   ├── routes/
│   │   └── propiedad.routes.ts ← 7 endpoints de Propiedad
```

```

├── asesor.routes.ts      ← 5 endpoints de Asesor
├── controllers/
│   ├── PropiedadController.ts
│   └── AsesorController.ts
├── repositories/
│   ├── PropiedadRepository.ts
│   └── AsesorRepository.ts
├── middlewares/validateId.ts
├── application/
│   ├── services/
│   │   ├── PropiedadService.ts
│   │   └── AsesorService.ts
│   ├── validations/
│   │   ├── propiedad.validation.ts
│   │   └── asesor.validation.ts
│   └── dtos/
│       ├── requests/      ← Create/Update/Filtros DTOs
│       └── responses/     ← PropiedadResponse, AsesorResponse
├── domain/
│   ├── entities/
│   │   ├── Propiedad.ts      ← PropiedadEntity, PaginatedResult
│   │   └── Asesor.ts        ← AsesorEntity
│   └── ports/
│       ├── IPropiedadRepository.ts ← Sin import Prisma
│       └── IAsesorRepository.ts
├── prisma/
│   ├── schema.prisma      ← 7 entidades, 5 enums
│   └── migrations/       ← init migration

```

Documentación ([docs/](#))

```

docs/
├── CONTEXTO-COMPLETO-PROYECTO.md ← Este archivo
├── DIAGRAMA-CLASE-ENTIDADES.md   ← Diagrama completo con Mermaid
├── TT-001-configuracion-monorepo.md
├── TT-002-esquema-prisma-modelo-datos.md
├── TT-003-conexion-postgresql-express.md
├── TT-004-crud-propiedades.md
├── TT-005-crud-asesores.md
└── TT-005.5-refactor-hexagonal.md

```

12. Próximos Pasos

Inmediato (Sprint 1)

1. **TT-006:** API de Mensajes + Cloudinary (Nodemailer, multer, Cloudinary SDK)
2. **TT-007:** Autenticación JWT (login, register, middleware)
3. **TT-008:** Búsqueda y filtros avanzados (text search, price range)

4. Frontend público por Jhonny (React + Vite)
5. Admin panel por Kelvin
6. Mapa interactivo por Allan
7. Chatbot GROQ por Ricardo

Sprint 2

- JWT auth, favoritos, reportes, citas, valoraciones, chat en vivo

Sprint 3

- Notificaciones push, exportar datos, Google Maps, tours virtuales, sugerencias IA, deploy, SEO, modo oscuro

13. Convenciones y Reglas

- **Nombres de archivos:** PascalCase para clases/components, camelCase para instancias/variables
- **Rutas API:** RESTful, plural (`/api/propiedades`, `/api/asesores`)
- **Status codes:** 201 (create), 200 (success), 204 (delete), 400 (validation), 404 (not found), 500 (error)
- **Manejo de errores:** Controller captura errores del Service, devuelve JSON con `message` y opcional `errors`
- **IDs:** UUID v4, generados por Prisma (`@default(uuid())`)
- **Fechas:** ISO 8601, manejadas por Prisma (`@default(now())`, `@updatedAt`)
- **Validación:** Zod en la capa de Service, antes de llamar al Repository
- **DTOs:** Tipos de entrada/salida explícitos, separados de las entidades de Prisma

Documento generado el 28/06/2026. Contiene el estado completo del proyecto Alpha Inmobiliaria para ser usado como contexto por asistentes de IA.